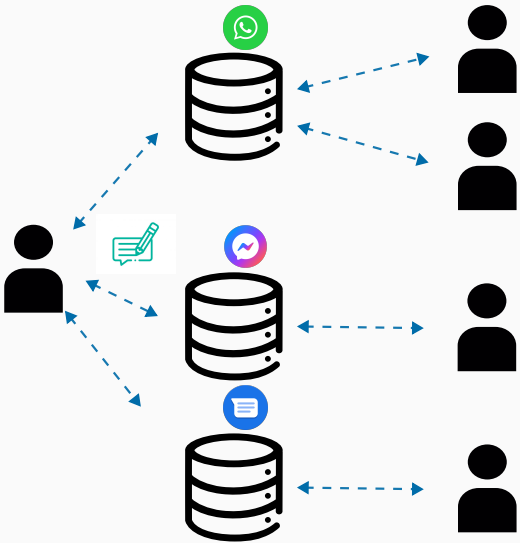
SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier, Ruben Taelman

October 20, 2026

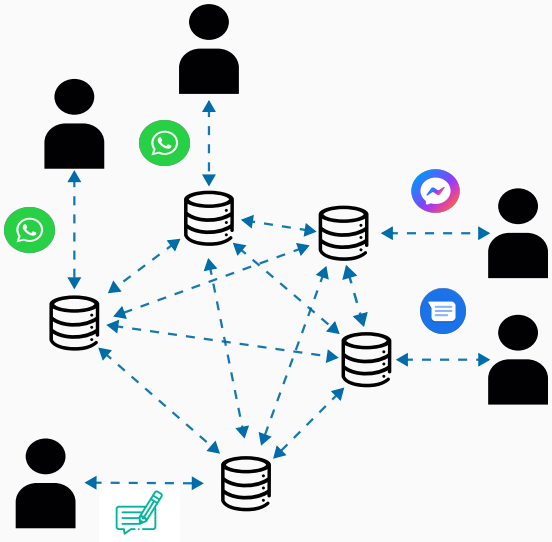
Department of Electronics and Information Systems, Ghent University – Imec

The Motivation for Decentralized Personal Data Storage



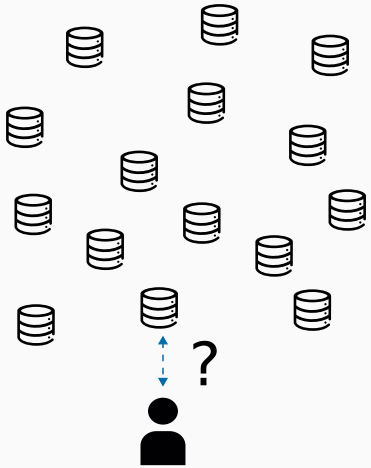
- Each application has its own data
- Stifles innovation
- Causes vendor lock-in

The Motivation for Decentralized Personal Data Storage



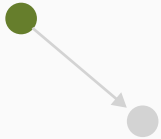
- Each application accesses a common decentralized data environment
- Easy to switch vendors
- Promotes innovation

Querying Structured Decentralized Environments



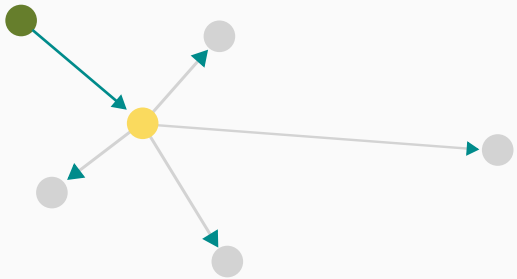
- How do we discover and query these sources?
- Use discovery-based querying, such as link traversal

Link Traversal: Seed Document



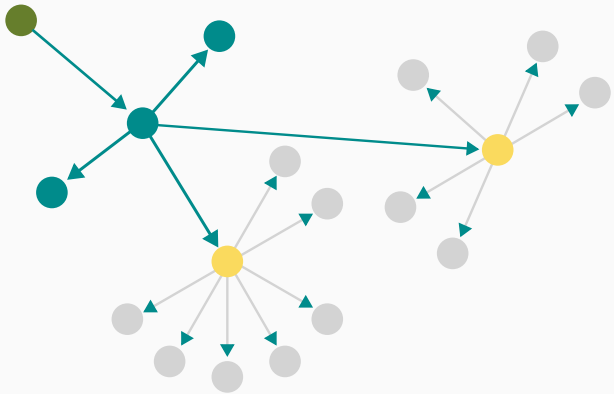
- Link Traversal starts from seed documents (URIs)
- These are provided by the user or specified in the query

Link Traversal: Traversal



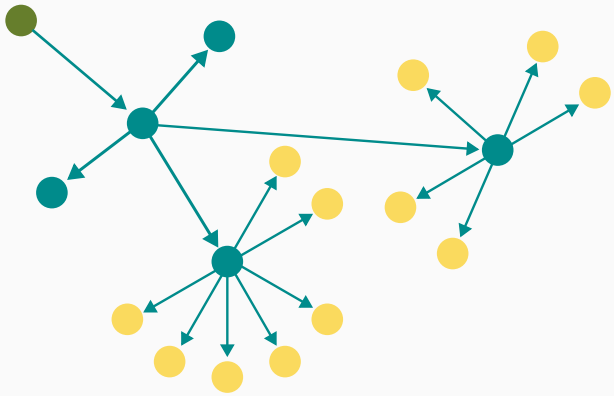
- New URIs are extracted from the seed document
- URIs are extracted in accordance with reachability criteria
- Uses the follow-your-nose principle

Link Traversal: Traversal



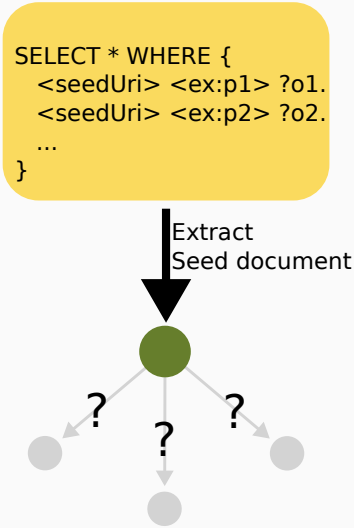
- New URIs are dereferenced and the process is repeated

Link Traversal: Termination



- This continues until all links are dereferenced
- Continuously produces results

Query Optimization for Link Traversal

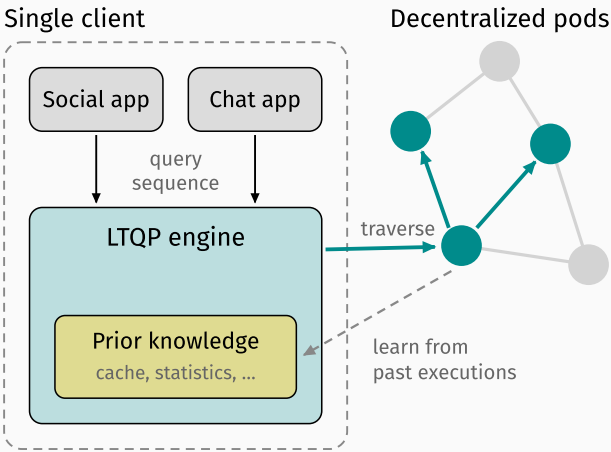


The query (partly) determines:

- The queried data
- The topology of the queried data
- The query-relevant documents

Result: Limited prior knowledge for query optimization

Query Optimization for Link Traversal: Client-side Engines

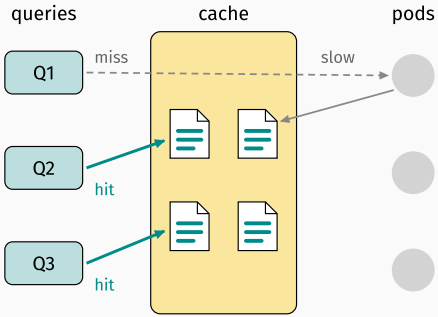


- Engine instances serve a single client
- Can we exploit client-specific usage patterns and ‘personalize’ optimization
- Gain prior knowledge by looking at past executions

Problem Statement

Evaluating the performance of client-specific optimization techniques in realistic, user-specific scenarios

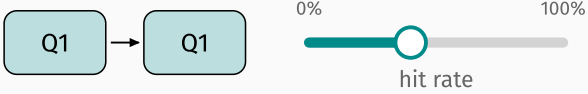
Evaluating Personalized Optimization through Caching



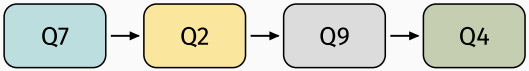
- Personalized optimization relies on data from preceding queries
- Caching is a direct, well-studied instance of this (Hartig 2011)
- *How does the caching literature evaluate such techniques?*

Existing Evaluation Approaches

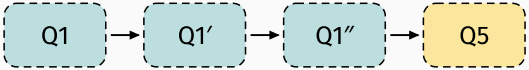
Micro benchmark



Random grouping



Real-world query log



- Micro benchmarks isolate single variables
- End-to-end benchmarks group unrelated queries or simulate simple patterns
- Handcrafted workloads are high quality, but small
- Real-world query logs do not exist for discovery-based querying

Goal

- Create a unified benchmark for client-specific approaches
- Simulate real-world user behavior in reproducible manner

Simulation Methodology: Base Benchmark

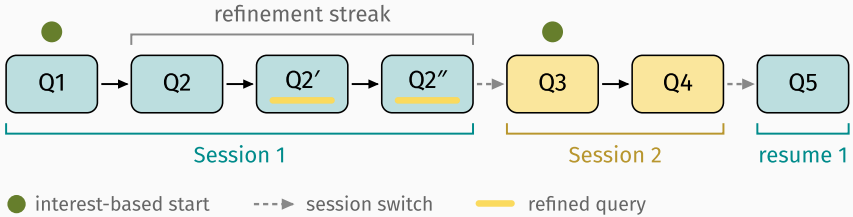
Simulated Solid Environment: SolidBench (Taelman and Verborgh 2023)

- Based on the realistic LDBC Social Network Benchmark
- Data is stored in vaults simulating social media application

Data attributes

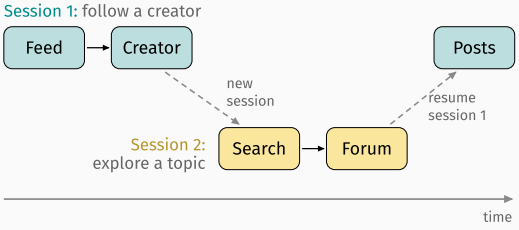
- Generates 158,233 RDF files
- Across 1,531 data vaults
- containing a total of 3,556,159 triples.

Simulation Methodology: Generating a Query Sequence



- Each sequence starts from a randomly selected base user
- Sessions chain dependent queries, refinement streaks modify them step-by-step
- A single seeded random generator keeps sequences reproducible

Usage Pattern: Logical Sessions



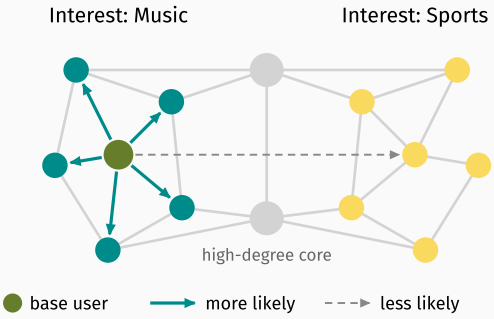
Observed (Meiss et al. 2009)

- Browsing groups into goal-oriented sessions
- New sessions in ∼15% of clicks

Simulated

- Next query instantiated from previous results
- Switch to a new session or resume an old one

Usage Pattern: User Interests



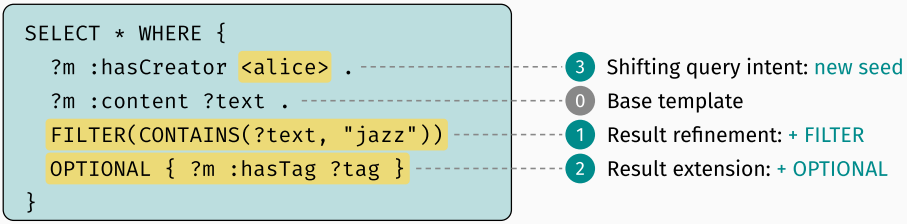
Observed (Mislove et al. 2007; Mitzlaff et al. 2013)

- Users form communities around shared interests
- Shared interests drive more interactions

Simulated

- User similarity on interests and studies
- Similar users are more likely to start a session

Usage Pattern: Query Refinement



Observed (Bonifati et al. 2020; Arenas et al. 2025)

- Queries refined step-by-step
- Mostly `FILTER`, `UNION`, `OPTIONAL`

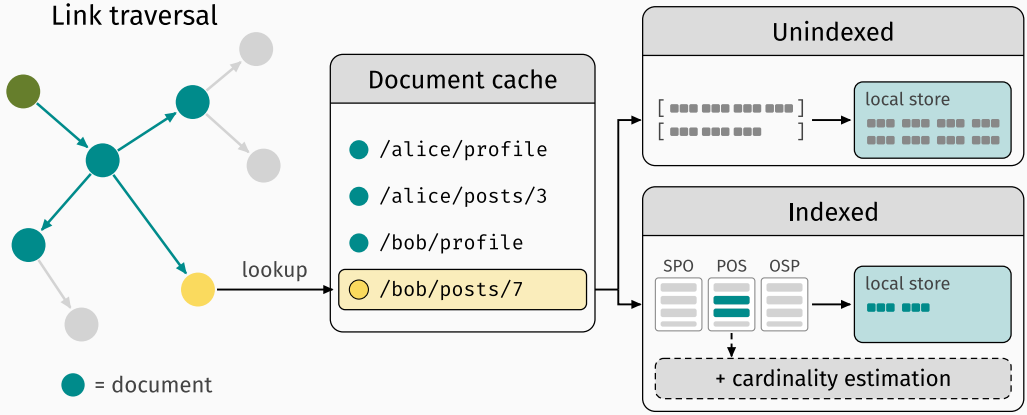
Simulated

- Composable transformations on queries
- Randomly select sequence of transformations to add to query sequence

Validating the Benchmark: Client-side Caching

- Performance of caching documents relies on data locality between queries
- Investigating cache performance reveals insights into our simulation
- Goal: validate the benchmark, not an exhaustive cache comparison

Caching Approaches



Implemented in Comunica, LRU eviction, three sizes: S (350k quads, ~10%), M (700k, ~20%), L (1.4M, ~40%)

Experimental Setup

Workload

- 8 generated sequences, 192 queries (discover and short templates)

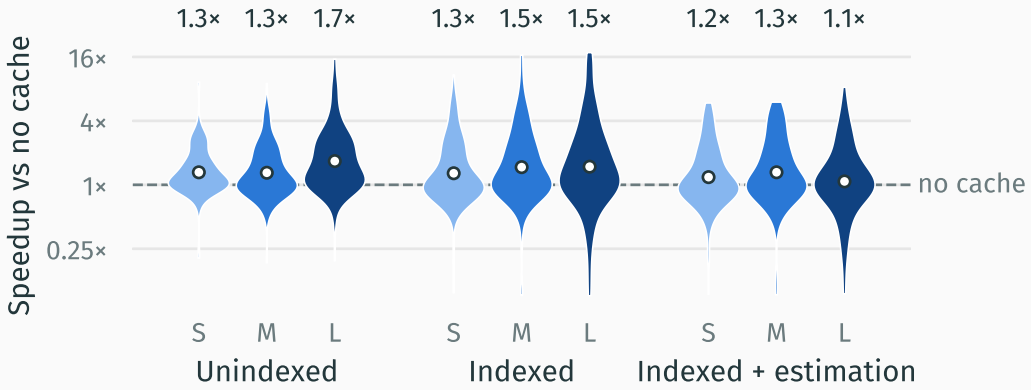
Execution

- Each sequence repeated 10 times, 70–90 ms injected network latency, 180 s timeout
- Cache scoped to a single sequence, no state shared between runs

Measured

- Cache hit rates, evictions, and speedup over no cache

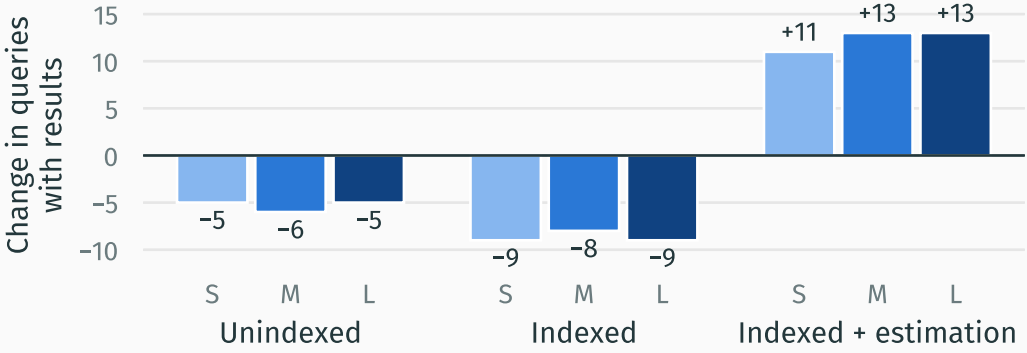
Results: Bigger Caches Help, Indexing Alone Does Not



Speedup of each query instance vs. the same instance without cache; marker: geometric mean

Large caches help most; indexed caches also slow some queries down

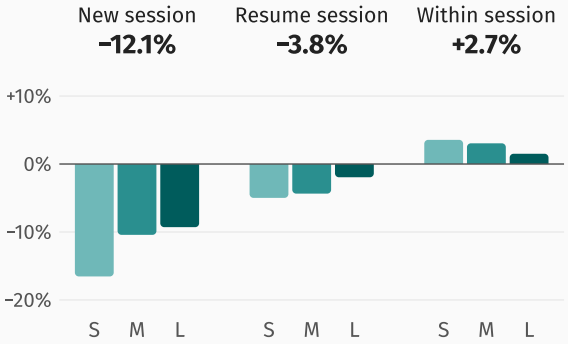
Results: Cardinality Estimation Lets More Queries Return Results



Query instances returning at least one result before the timeout, compared to no cache

Cardinality estimation gets results for 11–13 more queries

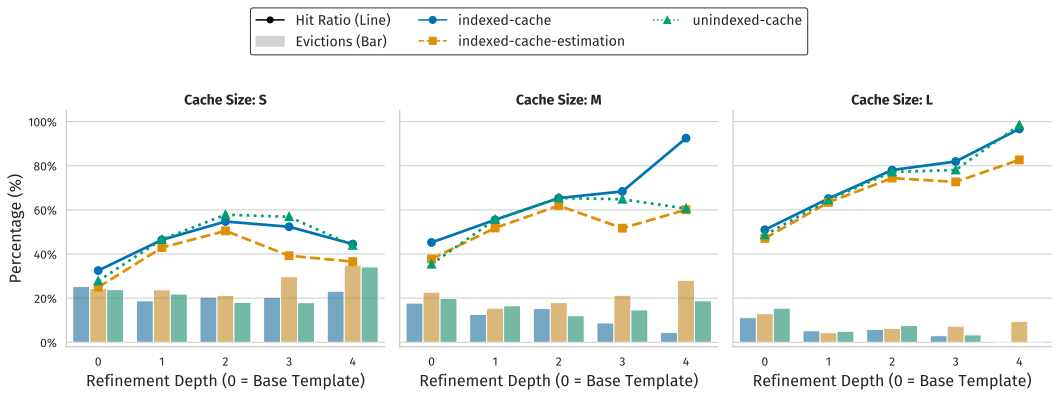
Results: Session Switching Costs Cache Hits



- Hit rate compared to the average of the same template
- Strongest for small caches

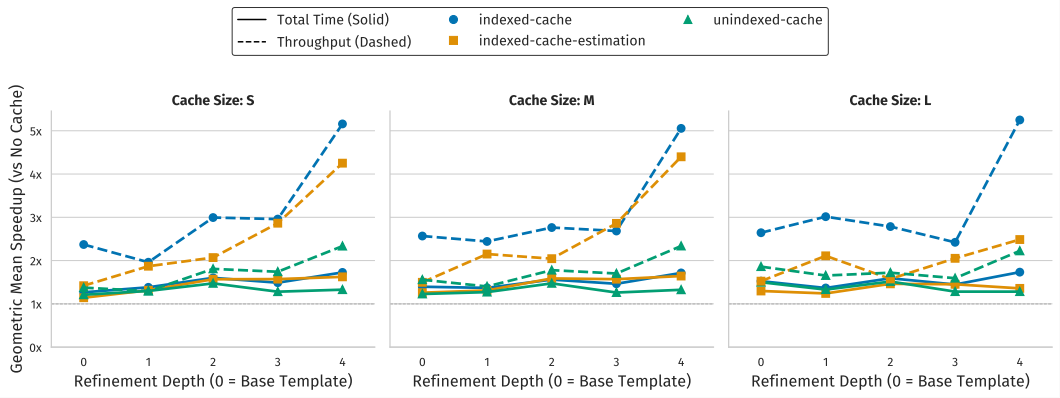
⇒ Cache sizing and eviction depend on session behavior

Results: Refinement Reuses Prior Data



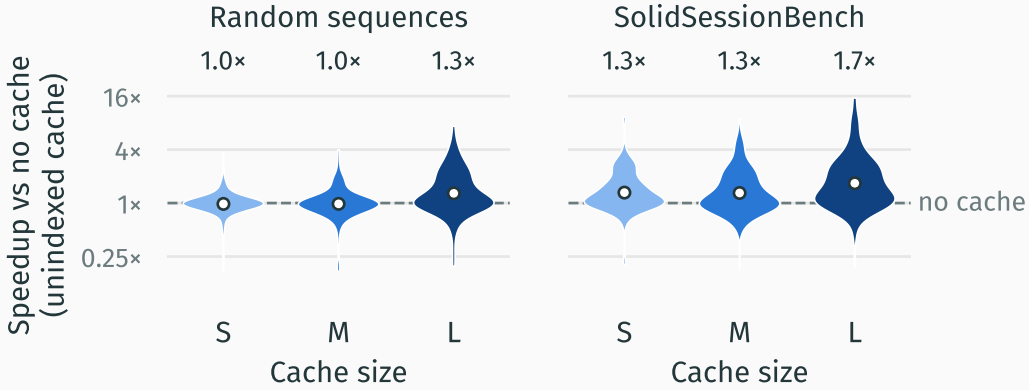
Hit rates rise with each refinement step, until small caches start thrashing

Results: Refinement Translates into Speedups



Up to ∼5× throughput at refinement depth 4 with the indexed cache

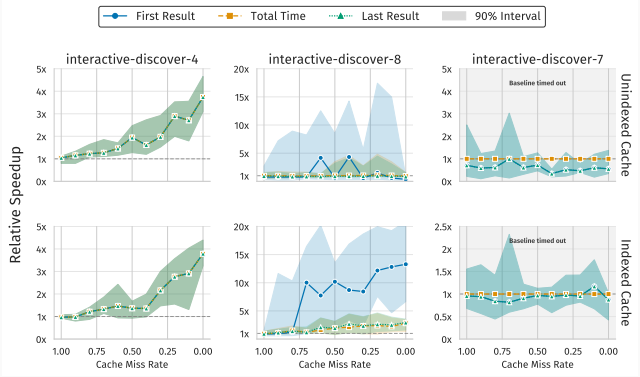
Comparison: Random Sequences Mislead



Speedup of each query instance vs. the same instance without cache; marker: geometric mean

Random: only large caches help vs. SolidSessionBench: small caches help too

Comparison: Micro Benchmarks Give Mixed Signals



- Each template run twice with simulated cache miss rates
- 8 of 12 templates speed up with hit rate
- Long-running queries: no speedup, or even slowdowns

⇒ Isolated queries do not show behavior in realistic sequences

Key Takeaways

- Sessions and refinements shape client-side performance
- Random sequences and micro benchmarks miss this

$\Rightarrow$ SolidSessionBench: realistic, reproducible query sequences

Future work

- Session-aware caching: sizing, indexing, and eviction tuned to sessions
- Reusing previous executions for query planning

Thank You! Questions?

SolidSessionBench

- Realistic sessions and refinements
- Reveals effects random sequences miss
- Reproducible: one seeded generator



github.com/SolidBench/SolidBench.js

References i

- Arenas, M. et al. (2025). **“Exploring Exploratory Querying”.** In: *Proceedings of the VLDB Endowment* 18.13, pp. 5731–5739.
- Bonifati, A., W. Martens, and T. Timm (2020). **“An analytical study of large SPARQL query logs”.** In: *The VLDB Journal* 29.2, pp. 655–679.
- Hartig, O. (2011). **“How Caching Improves Efficiency and Result Completeness for Querying Linked Data”.** In: *LDOW*.
- Meiss, M. et al. (2009). **“What’s in a session: tracking individual behavior on the web”.** In: *Proceedings of the 20th ACM conference on Hypertext and hypermedia*. Torino Italy: ACM, pp. 173–182. DOI: 10.1145/1557914.1557946.

References ii

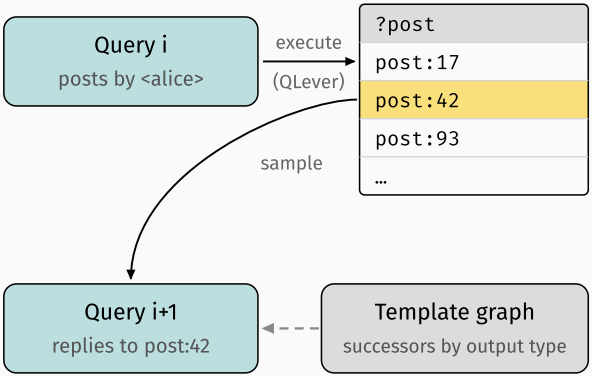
- Mislove, A. et al. (2007). **“Measurement and analysis of online social networks”.** In: *Proceedings of the 7th ACM SIGCOMM conference on Internet measurement*, pp. 29–42.
- Mitzlaff, F. et al. (2013). **“User-relatedness and community structure in social interaction networks”.** In: *arXiv preprint arXiv:1309.3888*.
- Taelman, R. and R. Verborgh (2023). **“Link traversal query processing over decentralized environments with structural assumptions”.** In: *International Semantic Web Conference*. Springer, pp. 3–22.

Simulation Methodology: Hyperparameters

Category	Parameter	Value
Log-normal (μ, σ)	Sequence length	3.0, 0.2
	Session length	1.7, 0.5
	Transition probability	-2.0, 0.25
Refinements	`patternProbability`	0.1
	`min/maxRefinementLength`	2, [3–5]
Instantiation	`temperature`	0.1
	`maxLogits`	5–10

A single seeded PRNG guarantees reproducible sequences

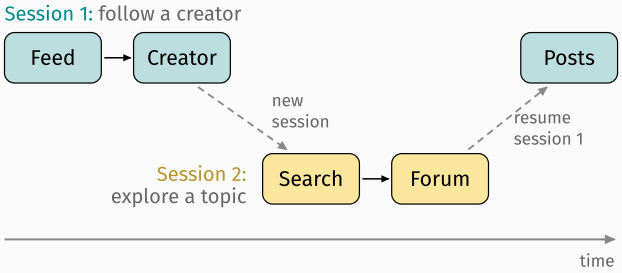
Simulation Methodology: Logical Query Sessions



No results → fall back to interest-based instantiation

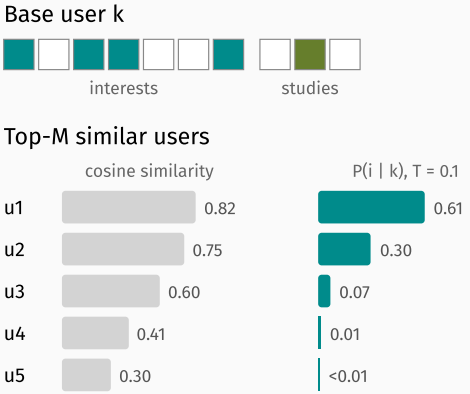
- Directed template graph: output types map to valid next templates
- Instantiation values sampled from previous results
- Simulates “click-through” behavior

Simulation Methodology: Session Switching



- Switch probability per sequence: log-normal, mean 15% (Meiss et al. 2009)
- Switch: new session or resume a previous one (50/50)
- No backtracking, balanced template distribution

Simulation Methodology: User Interest Modeling



- Binary vector of interests and studies per user
- Cosine similarity to base user k, keep top-M
- Softmax selects the first query of each session:

$$P(i \mid k) = \frac{e^{s_{k,i}/T}}{\sum_{u \in C_{k,M}} e^{s_{k,u}/T}}$$

Simulation Methodology: Refinement Patterns

Behavior	Transformation	Choke points
Result refinement	± `FILTER`, ± selective triple patterns	P.1, P.3
Result generalization	+ (non-)selective triple patterns	P.3, P.4
Result extension	+ triple patterns, + `OPTIONAL`/`UNION`	P.2–P.4
Query refinement	± `OPTIONAL`/`UNION`	P.2
Shifting query intent	New instantiation values (seeds)	T.1, T.2

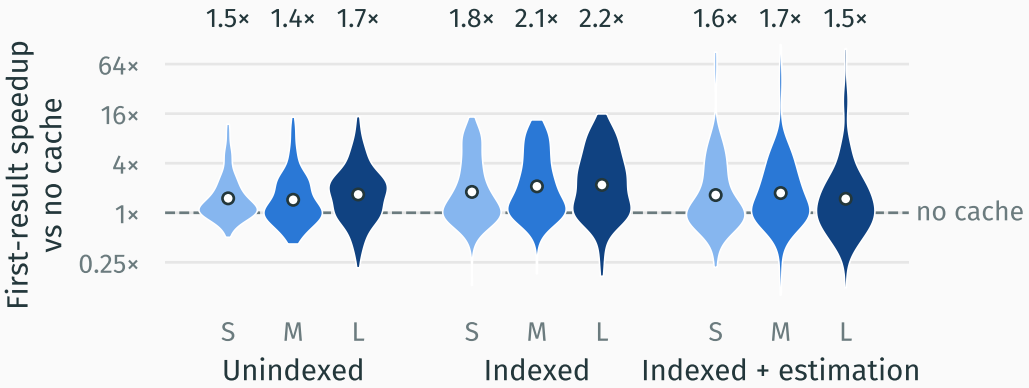
P.1 Filters
P.2 Filter push-down through `OPTIONAL`/`UNION`
P.3 Selective patterns
P.4 Non-selective patterns

Simulation Methodology: Refinements Impact Traversal

- **T.1** Predicate additions expand or contract the reachable sub-web
- **T.2** New starting points change the reachable documents
- **T.3** Changes the number of hops to relevant data (SolidBench L.1–L.6)
- **T.4** Alters usability of structural indexes, such as type indexes (L.8)

$\Rightarrow$ Custom refinement patterns can be defined in JSON

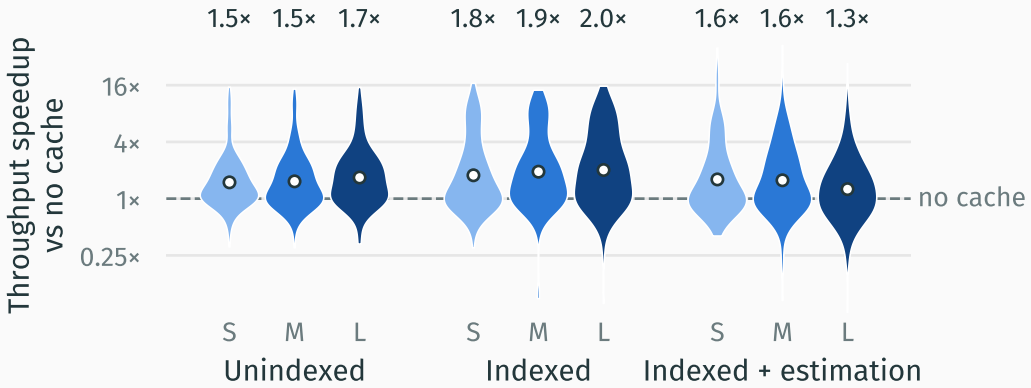
Results: Time to First Result



Instances returning results with and without cache; marker: geometric mean

Indexed caches deliver the first result ∼2× sooner on average

Results: Throughput



Results per second until the last result; marker: geometric mean

Indexed caches stream results fastest, but do not finish queries sooner